Efficient Sparse Matrix-Vector Multiplication with Various Optimization Methods

An-Che Liang, Guan-Ming Chiu, Pei-Yu Wu
National Taiwan University
Team 12

1 Environment

This section describes the hardware and software environment used for the experiments.

The experiments were run on an Intel(R) Xeon(R) Platinum 8352V CPU @ 2.10GHz machine with a topology of 2 sockets, 36 cores per socket, 2 threads per core, and 2 NUMA nodes. For OpenMP, the tested thread counts were 1, 2, 4, 8, and 16. Each point on the graph reports the median of 3 runs. The reported times are the `spmv_csr_openmp` execution times printed by the binaries, excluding file parsing and CSR construction.

2 Implementations and Applied Optimizations

The implementation study used three methodological levels. SPMV-NOT-CSR was treated as the reference baseline that evaluates sparse matrix-vector multiplication directly from unspecialized sparse input, so each row repeatedly traverses many irrelevant nonzeros. This baseline is useful for demonstrating correctness and for quantifying the cost of poor data locality and redundant work.

The next level, SPMV-SERIAL, applies a compressed sparse row organization before the compute phase. This changes the kernel from repeated global scans to row-local contiguous access, so each row processes only its own nonzeros. Methodologically, this isolates the effect of storage layout and access pattern improvements while keeping execution single-threaded.

SPMV-OPENMP extends the CSR methodology to parallel execution with several locality-conscious techniques. The CSR arrays are first-touch initialized in parallel (`schedule(static)`, `proc_bind(spread)`) so each thread's rows land on the local NUMA node. Each row is sorted by column index to improve sequential access into `x`, and for medium-size vectors a per-thread replica of `x` eliminates cross-socket reads. The kernel distributes rows with the same `schedule(static)` that matches the first-touch layout, and the inner loop uses `omp simd reduction` to exploit vector units. Thread placement defaults to `OMP_PROC_BIND=spread` and `OMP_PLACES=cores`.

3 Complexity Analysis of the Three Methods

Let R be the number of rows, N be the number of nonzeros, and p be the number of OpenMP threads.

For SPMV-NOT-CSR, each output row is computed by scanning the full sparse entry list, so the kernel performs approximately $R \times N$ comparisons/visits. Its time complexity is therefore $O(RN)$, which grows quickly even for moderately large matrices.

For SPMV-SERIAL, CSR preprocessing organizes entries by row so each nonzero is visited only once during the multiplication phase. The SpMV kernel complexity becomes $O(R + N)$, where the R term corresponds to per-row control work and the N term corresponds to nonzero multiply-accumulate operations.

For SPMV-OPENMP, the same CSR work is distributed across threads. Ignoring overhead, ideal parallel runtime is $O((R+N)/p)$. In practice, synchronization, thread startup, load imbalance across rows, and memory-bandwidth limits add overhead, so observed runtime is better represented as $O((R+N)/p+\alpha)$ with machine-dependent overhead term α.

In summary, the three methods move from redundant-work complexity ($O(RN)$), to work-efficient complexity ($O(R+N)$), to parallel work-efficient complexity with overhead ($O((R+N)/p+\alpha)$). This model explains why the largest gains come first from storage-format improvement, then from parallelism on larger workloads.

Table 1 summarizes the complexity trends of the three methods.

Method	Asymptotic Time
SPMV-NOT-CSR	$O(RN)$
SPMV-SERIAL	$O(R+N)$
SPMV-OPENMP	$O((R+N)/p+\alpha)$

Table 1: Complexity comparison of the three SpMV methods.

4 Correctness

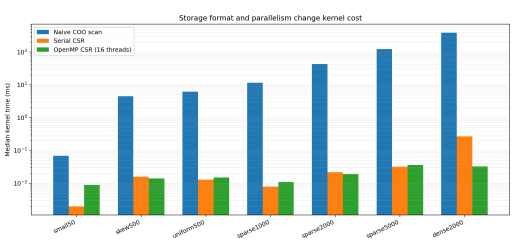
We wrote a custom bash script to validate correctness before collecting timing data. For each dataset, the script runs each binary and checks whether the program output contains `OK`, which indicates the computed result matches the provided `.gold` reference used by the checker inside the binaries. All benchmarked runs passed this check, so the performance comparisons below are between correct implementations.

5 How Does Storage Format Influence Correctness, Memory Usage, and Runtime?

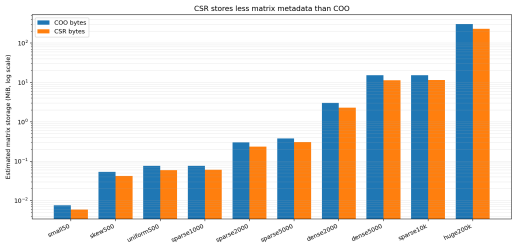
Correctness did not change across COO and CSR: all versions produced the expected output within the assignment tolerance. The real differences are memory layout and kernel cost.

Figure 1a shows the runtime gap directly. On `large_sparse_2000`, the naive COO scan took 43.31 ms while serial CSR took 0.0220 ms, a 1968.5x reduction. On `large_denserows_2000`, naive COO still needed 383.9 ms while serial CSR dropped to 0.2680 ms. The gap grows because CSR only touches the nonzeros that belong to the current row, while the naive baseline repeatedly rechecks unrelated entries.

Figure 1b shows the estimated matrix-storage footprint assuming 32-bit indices and 64-bit values; the y-axis is plotted in log scale so both small and large testcases are visible. Let N be the number of nonzeros and R be the number of rows. In COO, each nonzero stores a row index, a column index, and a value, so the footprint is approximately $N(4+4+8)=16N$ bytes. In CSR, each nonzero stores one column index and one value, plus a row-pointer array, so the footprint is approximately $N(4+8)+4(R+1)=12N+4(R+1)$ bytes. For large matrices this is consistently smaller. For example, huge_200k_100 is about 305.2 MiB in COO versus 229.6 MiB in CSR.



(a) Storage runtime comparison



(b) Estimated matrix storage (y-axis in log scale)

6 How Does Matrix Size Influence Performance?

Matrix size does influence performance, but it is not the only factor. As our complexity analysis shows, the runtime is proportional to $(R + N)$. Among the testcases provided, the dominant trend is that runtime scales with the amount of nonzero work, not just the matrix size. Figure 2 orders the matrices by `nnz`, and both CSR implementations follow that growth. Tiny matrices stay below 0.1 ms, the 1M-nnz matrices land around the sub-millisecond to low-millisecond range, and the 20M-nnz `huge_200k_100` case is the clear outlier.

On `large_10000_sparse_100` (1,000,000 nonzeros), serial CSR needed 1.604 ms. On `huge_200k_100` (20,000,000 nonzeros), serial CSR rose to 38.41 ms. The 20x increase in nonzeros therefore produced roughly a 23.9x increase in kernel time, which is close to the work increase expected for SpMV.

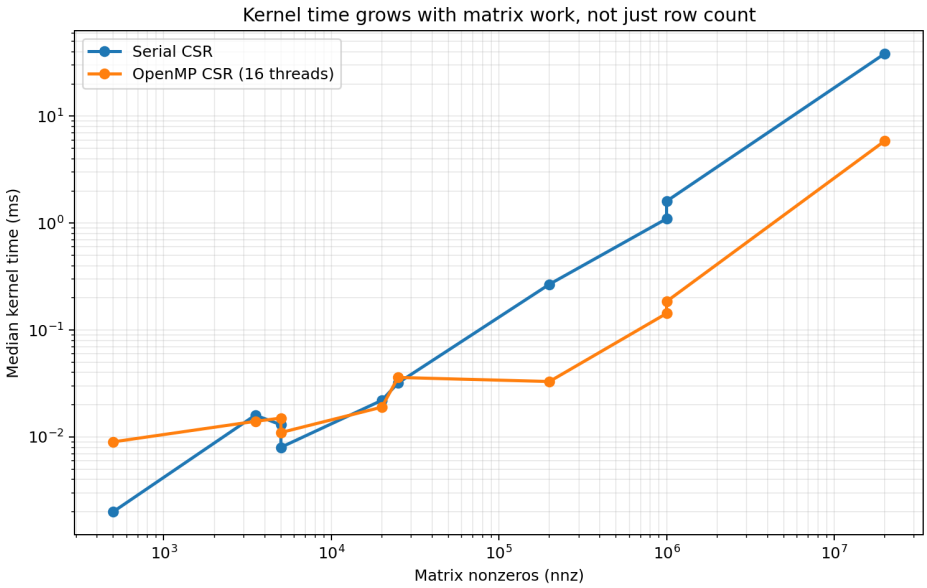


Figure 2: Matrix size vs runtime

7 How Does the Number of Threads Influence Performance?

The scaling behavior is strongly workload-dependent.

Table 2 first reports the measured median runtimes (ms) for two representative datasets across thread counts.

Threads	`large_sparse_5000`	`huge_200k_100`
1	0.055	36.763
2	0.066	33.348
4	0.035	21.076
8	0.025	11.403
16	0.036	5.864

Table 2: OpenMP thread-scaling data (median runtime in ms).

For `large_sparse_5000`, the table shows limited net benefit: runtime changes from 0.055 ms (1 thread) to 0.036 ms (16 threads), with non-monotonic behavior at intermediate thread counts, indicating that overhead and memory effects dominate quickly for this small workload. In contrast, for `huge_200k_100`, the table shows clear scaling, with runtime dropping from 36.763 ms (1 thread) to 5.864 ms (16 threads) and progressively larger gains as more cores are used.

In other words, more threads help when there is enough nonzero work and memory-level parallelism to keep the cores busy. On small matrices, parallel overhead dominates sooner.

8 What Is the Speedup of the Parallel Code Compared to the Serial Version?

The best 16-thread speedup in these experiments was on `large_10000_sparse_100` at 8.67x over serial CSR. The weakest 16-thread speedup among the scaling datasets was on `large_sparse_5000` at 0.89x.

For the largest case, `huge_200k_100`, serial CSR took 38.41 ms and the 16-thread OpenMP kernel took 5.864 ms, which is a 6.55x speedup.

Figure 3 plots both runtime and speedup versus thread count. The ideal line is included as a reference; the measured curves stay below it because SpMV is memory-bound and the rows do not all carry the same amount of work.

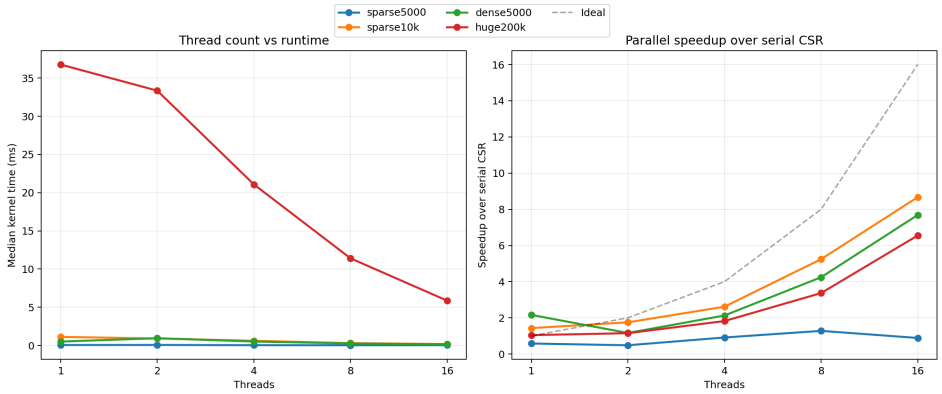


Figure 3: Thread scaling and speedup

9 Explain the Observed Scalability: Strong vs. Weak

Our experiments primarily measure strong scaling, not weak scaling. For each matrix, the problem size stays fixed while the thread count changes from 1 to 16. The plots therefore answer: how much faster does the same SpMV get when more threads are applied?

The results show moderate strong scaling on the larger matrices and weak strong scaling on the smaller ones. This is expected for SpMV because the kernel has low arithmetic intensity and quickly becomes limited by memory bandwidth, NUMA traffic, and synchronization overhead. The OpenMP implementation improves strong scaling by preserving locality: static row partitioning aligns with the first-touch initialization, thread pinning spreads threads across cores, and the optional thread-local copy of the input vector reduces shared-vector contention for medium-size inputs.

A true weak-scaling study would increase matrix size proportionally with thread count so that each thread keeps roughly constant work. That was not the experiment required by the assignment questions here, so we describe the observed behavior as strong scaling.

10 Additional Insights

Comparing `large_sparse_2000` and `large_denserows_2000` shows why nnz distribution matters: although they have the same row count, the denser matrix has 10x more nonzeros and correspondingly higher kernel time. Finally, the OpenMP version is also not just "serial CSR plus threads"; its NUMA-aware first-touch and thread-pinning strategy matches the dual-socket machine configuration, which helps maintain effectiveness on larger matrices.